

```
!pip install -q -U google-generativeai gradio
```

```
===== 24.2/24.2 MB 28.2 MB/s eta 0:00:00
===== 56.0/56.0 kB 1.9 MB/s eta 0:00:00
```

```
import google.generativeai as genai
GOOGLE_API_KEY = 'AIzaSyBiPcb7aFTBOYik-qfe7pB-ZsVCfp5Zpes'
genai.configure(api_key=GOOGLE_API_KEY)
```

```
print("API Configured Successfully!")
```

```
API Configured Successfully!
/usr/local/lib/python3.12/dist-packages/google/colab/_import_hooks/_hook_injector.py:55: FutureWarning:
```

All support for the `google.generativeai` package has ended. It will no longer be receiving updates or bug fixes. Please switch to the `google.genai` package as soon as possible. See README for more details:

<https://github.com/google-gemini/deprecated-generative-ai-python/blob/main/README.md>

```
loader.exec_module(module)
```

```
SYSTEM_PROMPT = (
    "You are the 'Petition Intake Assistant'. Your goal is to collect exactly 6 pieces of information "
    "from the user to file a formal petition: \n"
    "1. Full Name\n"
    "2. Phone Number\n"
    "3. Area/Location\n"
    "4. Petition Title\n"
    "5. Category (e.g., Water, Roads, Electricity, Safety)\n"
    "6. Detailed Description\n\n"
    "RULES:\n"
    "- If the user just says 'Hi' or 'Help me', greet them and ask for their Name and Area to start.\n"
    "- Ask for missing information one or two items at a time so it feels like a conversation.\n"
    "- Once you have ALL 6 pieces of information, generate a FINAL FORMAL PETITION draft "
    "using all the details provided. Confirm with the user if they want to submit it."
)
```

```
def chat_engine(message, history, state):
    # Initialize state if it's the first message
    if state is None:
        state = {"mode": None}

    # 1. Handle Button Clicks / Intent
    if message == "File your complaints":
        state["mode"] = "collecting"
        return "I'm ready to help you file a complaint. To start, what is your **Full Name** and **Phone Number**?", state

    if message == "Any other related":
        state["mode"] = "general"
        return "I can answer questions about existing petitions or how the process works. What is your question?", state

    # 2. Logic for Data Collection (The 6 items)
    if state["mode"] == "collecting":
        # We send the system prompt + history to Gemini to figure out what's missing
        full_context = f"{SYSTEM_PROMPT}\n\nChat History: {history}\nUser says: {message}"

        # Using the generative_ai model you configured in Cell 2
        model = genai.GenerativeModel('gemini-1.5-flash')
        response = model.generate_content(full_context)
        return response.text, state

    # 3. Logic for General Queries
    else:
        model = genai.GenerativeModel('gemini-1.5-flash')
        response = model.generate_content(f"Answer this query professionally: {message}")
        return response.text, state
```

```
import gradio as gr
```

```
with gr.Blocks(theme=gr.themes.Soft()) as demo:
    gr.Markdown("# 📄 Petition Response Engine")
    gr.Markdown("Click a button below to start.")
```

```

# State variable to track progress
state = gr.State()

chatbot = gr.Chatbot(label="Petition Assistant")
msg = gr.Textbox(label="Type here...", placeholder="Click a button or type a message...")

with gr.Row():
    btn_file = gr.Button("📄 File your complaints", variant="primary")
    btn_info = gr.Button("ℹ️ Any other related")
    btn_submit = gr.Button("✅ Final Submit", variant="stop")

# Define how the chatbot responds
def respond(message, history, st):
    bot_message, new_st = chat_engine(message, history, st)
    history.append((message, bot_message))
    return "", history, new_st

# Button click actions
btn_file.click(respond, [gr.State("File your complaints"), chatbot, state], [msg, chatbot, state])
btn_info.click(respond, [gr.State("Any other related"), chatbot, state], [msg, chatbot, state])

# Textbox enter action
msg.submit(respond, [msg, chatbot, state], [msg, chatbot, state])

# Final Submit Button alert
btn_submit.click(lambda: gr.Info("Petition data has been locked and submitted to the engine!"))

# Launch the project
demo.launch(share=True, debug=True)

```

/tmp/ipython-input-791956154.py:3: UserWarning: The parameters have been moved from the Blocks constructor to the launch() method with gr.Blocks(theme=gr.themes.Soft()) as demo:

Colab notebook detected. This cell will run indefinitely so that you can see errors and logs. To turn off, set debug=False in launch.

* Running on public URL: <https://1744423b19136ed9b0.gradio.live>

This share link expires in 1 week. For free permanent hosting and GPU upgrades, run `gradio deploy` from the terminal in the workspace.

Next steps: [🚀 Deploy to Cloud Run](#)

Start coding or [generate](#) with AI.

